Python Machine Learning Cheat Sheet

---

1. Preprocessing

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# One-hot encode categorical columns
df = pd.get_dummies(df, drop_first=True).astype(int)

# Split into X (features) and y (target)
X = df.drop('target_column', axis=1)
y = df['target_column']

# Train-test split (stratify only for classification)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42)

# Feature scaling (especially for linear models, SVM, PCA, KMeans)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

---

2. Handling Class Imbalance (Classification Only)

```
from imblearn.over_sampling import SMOTE

# Balance training data
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
```

---

3. Supervised Learning Models

🟦 A. Classification (Nominal Target)

1. Random Forest

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_res, y_train_res)
y_pred = model.predict(X_test)
```

2. Logistic Regression

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train_res, y_train_res)
y_pred = model.predict(X_test_scaled)
```

3. XGBoost

```
from xgboost import XGBClassifier

model = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
model.fit(X_train_res, y_train_res)
y_pred = model.predict(X_test)
```

🟧 B. Regression (Numeric Target)

1. Random Forest Regressor

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

2. Linear Regression

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
```

3. XGBoost Regressor

```
from xgboost import XGBRegressor

model = XGBRegressor()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

---

4. Unsupervised Learning Models

🟪 A. Clustering

1. K-Means Clustering

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_train_scaled)
labels = kmeans.labels_
```

2. DBSCAN (Density-Based)

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X_train_scaled)
```

🟧 B. Dimensionality Reduction

1. PCA (Principal Component Analysis)

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_train_scaled)
```

2. t-SNE (for visualization only)

```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X_train_scaled)
```

---

5. Model Evaluation

🟦 Classification

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

🟨 Regression

```
from sklearn.metrics import mean_squared_error, r2_score

print("MSE:", mean_squared_error(y_test, y_pred))
print("R²:", r2_score(y_test, y_pred))
```

🟪 Clustering

```
from sklearn.metrics import silhouette_score

print("Silhouette Score:", silhouette_score(X_train_scaled, labels))
```

---

6. Feature Importance (Tree-Based Models)

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

importances = model.feature_importances_  # works for RF and XGBoost
feat_series = pd.Series(importances, index=X.columns)
top_feats = feat_series.sort_values(ascending=False).head(10)

plt.figure(figsize=(8, 5))
sns.barplot(x=top_feats.values, y=top_feats.index)
plt.title("Top 10 Feature Importances")
plt.show()
```

---

7. Hyperparameter Tuning (GridSearchCV)

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20]
}

grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    param_grid, cv=5, scoring='accuracy')

grid.fit(X_train_res, y_train_res)
print("Best Parameters:", grid.best_params_)
```

Summary Table

Task	Method / Tool
Encode categoricals	pd.get_dummies()
Train/test split	train_test_split()
Scale features	StandardScaler()
Handle imbalance	SMOTE, RandomOverSampler
Classification models	RF, Logistic, XGBoost
Regression models	RF Regressor, Linear, XGBoost
Clustering models	KMeans, DBSCAN
Dimensionality reduction	PCA, t-SNE
Evaluate classification	accuracy_score, confusion_matrix
Evaluate regression	mean_squared_error, r2_score
Evaluate clustering	silhouette_score
Feature importance	.feature_importances_
Hyperparameter tuning	GridSearchCV()